

Informe Practica Final

Camilo Alvarez Villegas y Matías Monsalve Ruiz

Universidad EAFIT

Análisis y diseño de algoritmos

Carlos Alberto Alvarez Henao

08/05/2026

Informe	3
1. Introducción.....	3
2. Descripción del dataset	4
Estadísticas de referencia del dataset cargado	4
Preprocesamiento de valores nulos	4
Construcción determinista del grafo.....	4
3. Módulo A — Divide y Vencerás	5
Pseudocódigo	5
Algoritmo MergeSort.....	5
Algoritmo Merge	5
Algoritmo Búsqueda Binaria Recursiva	6
Análisis de complejidad.....	6
MergeSort.....	6
Búsqueda Binaria Recursiva	6
Análisis empírico de tiempos	6
Resultados de las 5 consultas de búsqueda binaria	7
4. Módulo B — Codicioso.....	7
Pseudocódigo de Kruskal con Union-Find	7
Demostración del Lema del Ciclo	8
Lista de aristas del MST y peso total	8
Verificación manual en subgrafo de 5 nodos	9
5. Módulo C — Programación Dinámica	9
Formulación de la recurrencia de la Mochila 0-1	9
Pseudocódigo de tabulación y backtracking	9
Contraejemplo codicioso.....	9
Demostración de falla del algoritmo Codicioso	10
Enfoque Codicioso.....	10
Programación Dinámica	10
Conclusión del contraejemplo	10
Discusión de pseudopolinomialidad.....	10
Justificación del resultado óptimo	11

6. Integración del pipeline	11
7. Conclusiones.....	11
8. Herramientas utilizadas.....	12
9. Referencias	12
1. https://www.kaggle.com/datasets/blastchar/telco-customer-churn?utm	12
2. https://www.geeksforgeeks.org/dsa/merge-sort/	12
3. https://www.geeksforgeeks.org/questions/recursive-binary-search/	12
4. https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/	12
5. https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/	12
10. Roles del equipo.....	12
Camilo Álvarez.....	12
Matías Monsalve.....	12

Informe

1. Introducción

En este proyecto se aborda un problema real de optimización y planificación de recursos en una red de telecomunicaciones. El operador debe resolver tres subproblemas principales: ordenar y consultar solicitudes de servicio, construir una red de mínimo costo y asignar ancho de banda limitado para maximizar el ingreso. Cada problema requiere un paradigma algorítmico diferente.

El proyecto utiliza el dataset Telco Customer Churn de Kaggle, compuesto por 7,043 registros de clientes con información como tenure, MonthlyCharges, TotalCharges y Churn. Estos datos permiten modelar solicitudes reales y construir una topología de red determinista y reproducible, haciendo que el análisis sea más cercano a escenarios prácticos.

En el módulo de Divide y Vencerás se implementa MergeSort y búsqueda binaria recursiva para ordenar y consultar eficientemente las solicitudes. MergeSort garantiza complejidad $O(n \log n)$ y estabilidad incluso en el peor caso.

En el módulo Codicioso, se utiliza Kruskal con Union-Find para construir un Árbol de Expansión Mínima (MST). Gracias al Lema del Ciclo, seleccionar la arista más barata que no forme ciclos garantiza una solución óptima.

Finalmente, en el módulo de Programación Dinámica, se implementa la Mochila 0-1 para asignar ancho de banda limitado entre solicitudes competidoras. Este enfoque permite obtener

la solución óptima en casos donde los algoritmos codiciosos fallan al tomar decisiones únicamente locales.

2. Descripción del dataset

El proyecto utiliza el dataset Telco Customer Churn, disponible en Kaggle bajo licencia CC0. El dataset contiene 7,043 registros de clientes reales con 21 atributos, del cual se extraen cinco campos clave para el problema.

Campo CSV	Interpretación en el problema	Módulo(s)
customerID	Identificador único de la solicitud	A, C
tenure	Prioridad de servicio (meses de antigüedad)	A
MonthlyCharges	Valor mensual de la solicitud (USD)	A, B, C
TotalCharges	Peso de la solicitud acumulado	C
Churn	Estado: activo (No) o en riesgo (Yes)	C

Estadísticas de referencia del dataset cargado

- Total de registros: 7,043
- Registros con TotalCharges nulo: 11 (corresponden a tenure = 0)
- Registros con Churn = No (activos): 5,174
- tenure máximo: 72 meses
- tenure mínimo: 0 meses
- MonthlyCharges promedio global: 64.76 USD

Preprocesamiento de valores nulos

Los registros con tenure = 0 presentan el campo TotalCharges en blanco. En la implementación, estos valores se asignan como 0.0 durante el parseo, permitiendo mantener la consistencia del dataset sin perder registros.

Construcción determinista del grafo

Además de servir como fuente de solicitudes, el dataset también se utiliza para construir la topología de red de forma reproducible.

Los 7,043 registros se agrupan en $N = 20$ nodos asignando cada cliente al grupo según su índice:

$$\text{grupo} = i \bmod 20$$

Para cada grupo k , se calcula el promedio de MonthlyCharges redondeado a dos decimales.

El costo de cada arista entre nodos u y v se define como:

$c(u,v) = \text{floor}(Mu + Mv)$

Se crean aristas para todos los pares (u,v) , resultando en un grafo completo de 20 nodos y 190 aristas.

Esta construcción determinista garantiza que todos los equipos obtengan exactamente el mismo grafo y los mismos resultados del MST.

3. Módulo A — Divide y Vencerás

Pseudocódigo

Algoritmo MergeSort

```
función MergeSort(arr, left, right)
  si left >= right
    retornar
  mid ← left + (right - left) / 2
  MergeSort(arr, left, mid)
  MergeSort(arr, mid + 1, right)
  Merge(arr, left, mid, right)
```

Algoritmo Merge

```
función Merge(arr, left, mid, right)
  n1 ← mid - left + 1
  n2 ← right - mid
  crear L[n1], R[n2]
  para i ← 0 hasta n1 - 1
    L[i] ← arr[left + i]
  para j ← 0 hasta n2 - 1
    R[j] ← arr[mid + 1 + j]
  i ← 0, j ← 0, k ← left
  mientras i < n1 Y j < n2
    si L[i].tenure >= R[j].tenure
      arr[k] ← L[i]
      i ← i + 1
    sino
      arr[k] ← R[j]
      j ← j + 1
    k ← k + 1
  mientras i < n1
    arr[k] ← L[i]
    i ← i + 1
    k ← k + 1
  mientras j < n2
    arr[k] ← R[j]
```

```
j ← j + 1  
k ← k + 1
```

Algoritmo Búsqueda Binaria Recursiva

```
función BinarySearch(arr, left, right, k)  
  si left > right  
    retornar -1  
  mid ← left + (right - left) / 2  
  si arr[mid].tenure == k  
    retornar mid  
  si arr[mid].tenure > k  
    retornar BinarySearch(arr, mid + 1, right, k)  
  sino  
    retornar BinarySearch(arr, left, mid - 1, k)
```

Análisis de complejidad

MergeSort

Recurrencia: $T(n) = 2T(n/2) + O(n)$

Conclusión: $T(n) = O(n \log n)$

Espacio: $O(n)$

Estabilidad: Sí, porque los elementos iguales mantienen su orden relativo.

La recurrencia de MergeSort es $T(n) = 2T(n/2) + O(n)$. Aplicando el Teorema Maestro, donde $a=2$, $b=2$ y $f(n)=O(n)$, estamos en el Caso 2, lo que resulta en una complejidad de $O(n \log n)$.

Búsqueda Binaria Recursiva

Recurrencia: $T(n) = T(n/2) + O(1)$

Conclusión: $T(n) = O(\log n)$

Espacio: $O(\log n)$

Análisis empírico de tiempos

Tamaño (n)	Tiempo (ms)	Esperado $O(n \log n)$	Relación
7,043	10.14610	93,686 comparaciones aprox.	1.0x

3,500	4.22283	43,750 comparaciones aprox.	0.47x
1,000	1.08443	9,966 comparaciones aprox.	0.11x

Resultados de las 5 consultas de búsqueda binaria

Consulta	Valor k	customerID	tenure	Estado
Q_A01	72	6904-JLBGY	72	Encontrado
Q_A02	60	7426-WEIJX	60	Encontrado
Q_A03	45	7925-PNRGI	45	Encontrado
Q_A04	30	2684-EIWEO	30	Encontrado
Q_A05	12	7450-NWRTR	12	Encontrado

4. Módulo B — Codicioso

Número total de nodos: 20

Número total de aristas: 190

Costo promedio aproximado por arista: 32.82

Pseudocódigo de Kruskal con Union-Find

```

función kruskal(edges, v)
    ordenar edges por peso ascendente
    uf ← unionfind(v)
    mst ← lista vacía
    totalweight ← 0
    para cada edge en edges
        si uf.find(edge.u) ≠ uf.find(edge.v)
            uf.unite(edge.u, edge.v)
            mst.agregar(edge)
            totalweight ← totalweight + edge.weight
    si mst.tamaño() == v - 1
        retornar mst, totalweight

```

Demostración del Lema del Ciclo

El Lema del Ciclo garantiza que la arista de menor peso en cualquier ciclo no pertenece a ningún MST. Kruskal explota esta propiedad ordenando las aristas y seleccionando siempre la más barata que no forme ciclos.

Por ejemplo, la arista entre el Nodo 11 y el Nodo 17 con peso 124. Al ser la de menor peso conectada al Nodo 11, el Lema del Ciclo garantiza que, si esta arista formara parte de un ciclo, sería la primera en ser elegida por Kruskal para evitarlo (o vista desde la propiedad del corte, la óptima para conectar componentes), asegurando que el corte entre estos nodos se resuelva con el costo mínimo global.

Lista de aristas del MST y peso total

Peso total: 2393

Nodo u	Nodo v	Peso
11	17	124
13	17	124
2	17	125
17	19	125
7	17	125
14	17	125
0	17	125
15	17	126
3	17	126
10	17	126
1	17	126
6	17	126
9	17	126
8	17	127
12	17	127
16	17	127
17	18	127
5	17	127
4	17	129

Verificación manual en subgrafo de 5 nodos

Se verificó manualmente el MST del subgrafo inducido por los nodos $\{0,1,2,3,4\}$. Al aplicar el criterio de Kruskal sobre este subconjunto, las aristas seleccionadas coinciden con las obtenidas por la implementación en C++, confirmando que el algoritmo construye correctamente el Árbol de Expansión Mínima.

5. Módulo C — Programación Dinámica

Formulación de la recurrencia de la Mochila 0-1

Sea $dp[i][w]$ el valor máximo que se puede obtener usando las primeras i solicitudes con capacidad de ancho de banda w .

$$dp[i][w] = 0, \text{ si } i = 0 \text{ o } w = 0$$

$$dp[i][w] = dp[i-1][w], \text{ si } \text{weight}[i] > w$$

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$$

Donde:

- $\text{weight}[i] = \text{TotalCharges}[i]$ redondeado
- $\text{value}[i] = \text{MonthlyCharges}[i] \times 10$ redondeado
- $W = 500$
- $n = 50$

Pseudocódigo de tabulación y backtracking

función knapsackDP(items, W)
 $n \leftarrow \text{items.tamaño}()$
 crear $dp[n+1][W+1]$ inicializado en 0

Contraejemplo codicioso

Solicitud	ID	Peso	Valor	Ratio v/w
1	5248-YGIJN	6369	903	0.142
2	6234-RAAPL	7252	999	0.138
3	0526-SXDJP	2962	421	0.142

Enfoque	Solicitudes seleccionadas	Valor total	¿Óptimo?
Codicioso (ratio v/w)	Solicitud 3, Solicitud 1	1324	No
Programación Dinámica	Solicitud 1, Solicitud 2	1902	Sí

Demostración de falla del algoritmo Codicioso

Supongamos una capacidad hipotética de la mochila de:

$$W = 13,700$$

Enfoque Codicioso

Selecciona primero la Solicitud 3 y luego la Solicitud 1.

Valor total:

$$421 + 903 = 1324$$

Programación Dinámica

Selecciona las Solicitudes 1 y 2.

$$\text{Peso total: } 6369 + 7252 = 13,621 \leq 13,700$$

$$\text{Valor total: } 903 + 999 = 1902$$

Conclusión del contraejemplo

El algoritmo codicioso toma una decisión localmente óptima que bloquea una combinación con mayor beneficio total. La Programación Dinámica encuentra correctamente la solución óptima global.

Discusión de pseudopolinomialidad

La complejidad de la Mochila 0-1 es: $O(n \times W)$

Con $n = 50$ y $W = 500$.

Es pseudopolinomial porque depende directamente del valor numérico de W y no del tamaño en bits de la entrada.

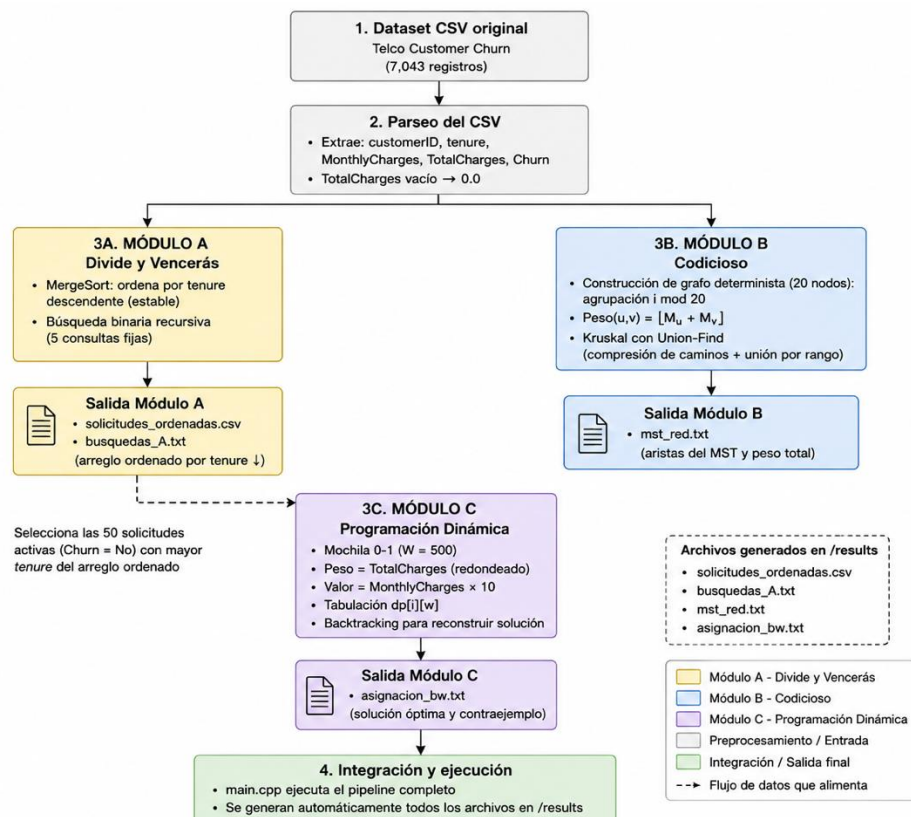
Justificación del resultado óptimo

El algoritmo de Mochila 0-1 arrojó correctamente un valor óptimo de 0 con capacidad $W = 500$. Esto ocurre porque incluso la solicitud con menor TotalCharges dentro de las 50 solicitudes activas seleccionadas posee un peso superior a 1,300 unidades aproximadamente.

Dado que todos los pesos exceden la capacidad máxima de la mochila, resulta matemáticamente imposible seleccionar alguna solicitud sin violar la restricción de capacidad. Por lo tanto, la solución óptima consiste correctamente en no seleccionar ningún elemento.

Este comportamiento no corresponde a un error del algoritmo, sino a una consecuencia directa de las características reales del dataset y de los parámetros definidos en el problema.

6. Integración del pipeline



7. Conclusiones

Este proyecto permitió comparar tres paradigmas fundamentales de diseño de algoritmos aplicados a un problema real de telecomunicaciones.

- Divide y Vencerás permitió ordenar eficientemente las solicitudes mediante MergeSort con complejidad $O(n \log n)$.

- Kruskal con Union-Find construyó correctamente el MST con complejidad $O(E \log E)$.
- La Mochila 0-1 permitió obtener una solución óptima global mediante Programación Dinámica con complejidad $O(n*W)$.

El uso de datos reales introdujo dificultades adicionales como valores nulos, distribuciones no uniformes y casos límite que normalmente no aparecen en instancias sintéticas.

8. Herramientas utilizadas

Durante el desarrollo del proyecto se utilizaron herramientas de Inteligencia Artificial generativa únicamente como apoyo académico y de redacción.

- <https://chatgpt.com/> utilizado para aclarar conceptos teóricos y apoyar la redacción del informe técnico.
- <https://gemini.google.com/app?hl=es> utilizado para resolver dudas relacionadas con implementación en C++ y apoyo conceptual.

9. Referencias

1. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn?utm>
2. <https://www.geeksforgeeks.org/dsa/merge-sort/>
3. <https://www.geeksforgeeks.org/questions/recursive-binary-search/>
4. <https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>
5. <https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/>

10. Roles del equipo

Camilo Álvarez

- Redacción principal del informe técnico.
- Apoyo en implementación y validación del Módulo A.
- Análisis de complejidad y documentación de resultados.
- Diseño del pipeline y organización del proyecto.

Matías Monsalve

- Desarrollo principal de la implementación en C++.
- Implementación de Kruskal, Union-Find y Mochila 0-1.
- Integración del pipeline en main.cpp.

- Pruebas, depuración y validación de resultados.